

# SOFTWARE DESIGN DOCUMENT

## Design and Development of an AI-Powered Career Preparation Application for Graduating College Students

### *A Capstone Project Documentation*

Presented in Partial Fulfillment of the Requirements for the Degree of Bachelor of Science in Information Technology

### **Technology Stack**

Flutter • Node.js & Express.js • Firebase Firestore • Firebase Authentication • OpenAI API / Google Gemini API

Document Version: 1.0

Date: July 2026

Prepared by: Jenard Reyes, Robert Jeremy Bicera, Jayvee Uribe

Institution: Systems Plus Computer College, Quezon City

## Document Revision History

| Version | Date      | Description                                                        | Author       |
|---------|-----------|--------------------------------------------------------------------|--------------|
| 0.1     | June 2026 | Initial draft — system overview and architecture                   | Project Team |
| 0.5     | July 2026 | Added module design, database design, and process flow diagrams    | Project Team |
| 1.0     | July 2026 | Finalized UI design, API design, security design, and testing plan | Project Team |

## Table of Contents

---

# 1. System Overview

---

## 1.1 Introduction

The transition from academic life to professional employment is one of the most critical junctures in a college student's career. Graduating students, particularly those enrolled in Information Technology programs, are often confronted with challenges in preparing polished resumes, developing confidence in job interviews, and accurately gauging their readiness for the workplace. Traditional career preparation resources — career counseling offices, generic online guides, and static resume templates — are frequently limited in availability, personalization, and scalability.

The AI-Powered Career Preparation Application is a cross-platform mobile solution designed to close this gap by giving graduating college students on-demand, personalized, and intelligent career preparation support. The system leverages generative artificial intelligence (via the OpenAI API or Google Gemini API) to analyze resumes, conduct simulated job interviews, evaluate technical and soft skills, and generate individualized career recommendations. All of this is delivered through a Flutter-based mobile front end backed by a Node.js/Express.js REST API and a Firebase Firestore data layer, allowing the application to scale efficiently while remaining maintainable and secure.

This Software Design Document (SDD) presents the complete technical design of the system. It captures the architectural decisions, module-level designs, database schema, process flows, interface designs, application programming interfaces (APIs), and security mechanisms that will guide the implementation, testing, and deployment of the application.

## 1.2 Project Objectives

The general objective of this capstone project is to design and develop a mobile application that uses artificial intelligence to help graduating college students prepare for employment. Specifically, the project aims to:

1. Develop a cross-platform mobile application using Flutter that runs on both Android and iOS devices.
2. Design and implement an AI-powered Resume Analyzer that evaluates resume content, structure, and Applicant Tracking System (ATS) compatibility, and provides actionable feedback.
3. Build an AI Mock Interview module capable of simulating realistic job interview scenarios with real-time, AI-generated evaluation and feedback.
4. Implement a Skills Assessment module that measures a student's technical and soft-skill competencies relative to their target career path.
5. Design a Career Readiness Dashboard that consolidates resume, interview, and skills data into a single, easy-to-interpret readiness score.
6. Develop an AI Career Recommendation engine that suggests suitable job roles and upskilling paths based on the student's profile and performance data.
7. Establish a secure, scalable backend architecture using Node.js, Express.js, and Firebase to support real-time data synchronization and authentication.

8. Apply modern software engineering practices — including Agile development, layered architecture, and RESTful API design — to ensure the system is maintainable and extensible.

## 1.3 Scope and Limitations

### 1.3.1 Scope

The system is scoped as a student-facing mobile application supported by a lightweight administrative capability. It covers the following functional areas:

- User registration and authentication using Firebase Authentication (email/password and Google Sign-In).
- Creation and management of a student profile, including academic background, skills, and career goals.
- Upload and AI-based analysis of resumes in PDF or DOCX format.
- AI-driven mock interview simulations tailored to a selected job role or industry.
- Skills assessment covering both technical (e.g., networking, programming) and soft-skill (e.g., communication, teamwork) competencies.
- A consolidated Career Readiness Dashboard summarizing the student's overall preparedness.
- AI-generated career and upskilling recommendations based on aggregated user data.
- Basic administrative functions for managing assessment content and monitoring aggregate usage analytics.

### 1.3.2 Limitations

The following are recognized as outside the current scope of the system:

- The application does not function as a job board or direct employer-matching/recruitment platform.
- Video-based interview analysis (e.g., facial expression or eye-contact detection) is not included in the initial release; the mock interview module relies on text and/or voice-to-text input.
- The system does not guarantee employment outcomes; it is a preparatory and advisory tool.
- AI-generated feedback is probabilistic in nature and may occasionally require human judgment to interpret or validate, as with any large language model-based system.
- Offline functionality is limited to viewing previously cached data; AI-powered features require an active internet connection.

## 1.4 Target Users

| User Type                                 | Description                                                                                                                                                           |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Graduating College Student (Primary User) | Undergraduate students nearing completion of their degree program who need to prepare resumes, practice interviews, assess their skills, and receive career guidance. |

| User Type                                   | Description                                                                                                                                                                          |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| System Administrator                        | Authorized personnel (e.g., career services staff or capstone adviser) responsible for managing assessment question banks, monitoring system usage, and maintaining content quality. |
| Academic Institution (Indirect Stakeholder) | Colleges and universities that may adopt the application as a supplementary career-readiness tool for their graduating cohorts.                                                      |

## 1.5 Definition of Terms

| Term            | Definition                                                                                                                                     |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| API             | Application Programming Interface — a set of defined rules that allow the mobile client and backend server to communicate.                     |
| ATS             | Applicant Tracking System — software used by employers to automatically screen and rank resumes.                                               |
| Firestore       | A cloud-hosted, NoSQL, document-oriented database provided by Firebase.                                                                        |
| JWT / ID Token  | A signed token issued by Firebase Authentication used to verify a user's identity on each API request.                                         |
| LLM             | Large Language Model — the class of AI model (e.g., GPT, Gemini) used to power resume analysis, interview simulation, and recommendations.     |
| Readiness Score | A composite metric calculated from a student's resume, interview, and skills-assessment performance, representing overall career preparedness. |
| SDD             | Software Design Document — this document, describing the technical design of the system.                                                       |

## 2. Development Methodology

### 2.1 Overview of the Agile Methodology

The project adopts the Agile Software Development methodology, specifically following a Scrum-based framework. Agile was selected over traditional linear models (e.g., Waterfall) because the system integrates rapidly evolving AI APIs (OpenAI/Gemini) and requires continuous feedback from end users (graduating students) and the capstone adviser. Agile's iterative nature allows the development team to release working increments of the application every sprint, gather feedback, and adjust the design and feature set accordingly, thereby reducing the risk of building features that do not meet user needs.

The development process is organized into short, time-boxed iterations called sprints, each typically lasting two weeks. Every sprint produces a potentially shippable increment of the application, accompanied by sprint planning, daily stand-up coordination, sprint review, and sprint retrospective activities.

### 2.2 Agile Roles

| Role                | Responsibility                                                                          | Assigned To                   |
|---------------------|-----------------------------------------------------------------------------------------|-------------------------------|
| Product Owner       | Defines and prioritizes the product backlog based on user needs and adviser feedback.   | Capstone Adviser / Team Lead  |
| Scrum Master        | Facilitates ceremonies, removes blockers, and ensures the team follows Agile practices. | Rotating team member          |
| Development Team    | Designs, implements, tests, and documents system features.                              | Project Members               |
| End Users / Testers | Provide feedback during sprint reviews and usability testing.                           | Graduating Student Volunteers |

### 2.3 Agile Development Phases

The project follows the standard Agile/Scrum lifecycle, adapted to a capstone project timeline:

9. Requirements Gathering & Product Backlog Creation — Collection of functional and non-functional requirements through interviews with graduating students, review of related literature, and consultation with the capstone adviser; requirements are translated into a prioritized product backlog of user stories.
10. Sprint Planning — At the start of each sprint, the team selects backlog items to be completed, breaks them into tasks, and estimates effort using story points.
11. Design & Development (Sprint Execution) — Iterative coding of features, including UI screens, backend endpoints, database schema updates, and AI-integration logic.
12. Daily Stand-up — Brief daily synchronization meetings to report progress, plan the day's work, and identify blockers.

13. Sprint Review / Demo — Demonstration of completed features to the Product Owner and sample end users for validation and feedback.
14. Sprint Retrospective — Reflection on what went well, what did not, and process improvements for the next sprint.
15. Testing & Quality Assurance — Continuous unit, integration, and user-acceptance testing conducted throughout each sprint (see Section 13, Testing Plan).
16. Deployment / Release — Deployment of stable increments to a staging environment, culminating in a final release candidate for capstone defense and demonstration.

## 2.4 Proposed Sprint Breakdown

| Sprint                   | Duration | Key Deliverables                                                                                     |
|--------------------------|----------|------------------------------------------------------------------------------------------------------|
| Sprint 0 — Inception     | 2 weeks  | Requirements gathering, system design, environment setup, Firebase project configuration.            |
| Sprint 1                 | 2 weeks  | User Authentication module (registration, login, Firebase Auth integration) and User Profile module. |
| Sprint 2                 | 2 weeks  | Resume Analyzer module: file upload, text extraction, AI integration for resume feedback.            |
| Sprint 3                 | 2 weeks  | AI Mock Interview module: question generation, response evaluation, session reporting.               |
| Sprint 4                 | 2 weeks  | Skills Assessment module: question bank, scoring engine, results storage.                            |
| Sprint 5                 | 2 weeks  | Career Readiness Dashboard and AI Career Recommendations module.                                     |
| Sprint 6                 | 2 weeks  | UI/UX refinement, security hardening, performance optimization, integration testing.                 |
| Sprint 7 — Stabilization | 2 weeks  | User acceptance testing, bug fixing, deployment preparation, documentation finalization.             |

## 2.5 Why Agile Fits This Project

- AI model behavior (prompt design, response quality) often requires iterative tuning that aligns naturally with sprint-based refinement.
- Frequent sprint reviews allow real graduating students to test features early and often, improving usability before final release.
- Agile accommodates the parallel development of independent modules (Resume Analyzer, Mock Interview, Skills Assessment) by different sub-teams.
- The capstone timeline benefits from Agile's incremental delivery, ensuring a working prototype exists early for adviser evaluation rather than only at the end of the term.



### 3. System Architecture

The system follows a client-server architecture composed of four principal layers: the Presentation Layer (Flutter mobile application), the Application Layer (Node.js/Express.js REST API), the Data & Platform Services Layer (Firebase Firestore, Firebase Authentication, and Firebase Cloud Storage), and the AI Services Layer (OpenAI API / Google Gemini API). This separation of concerns allows each layer to be developed, tested, scaled, and maintained independently.

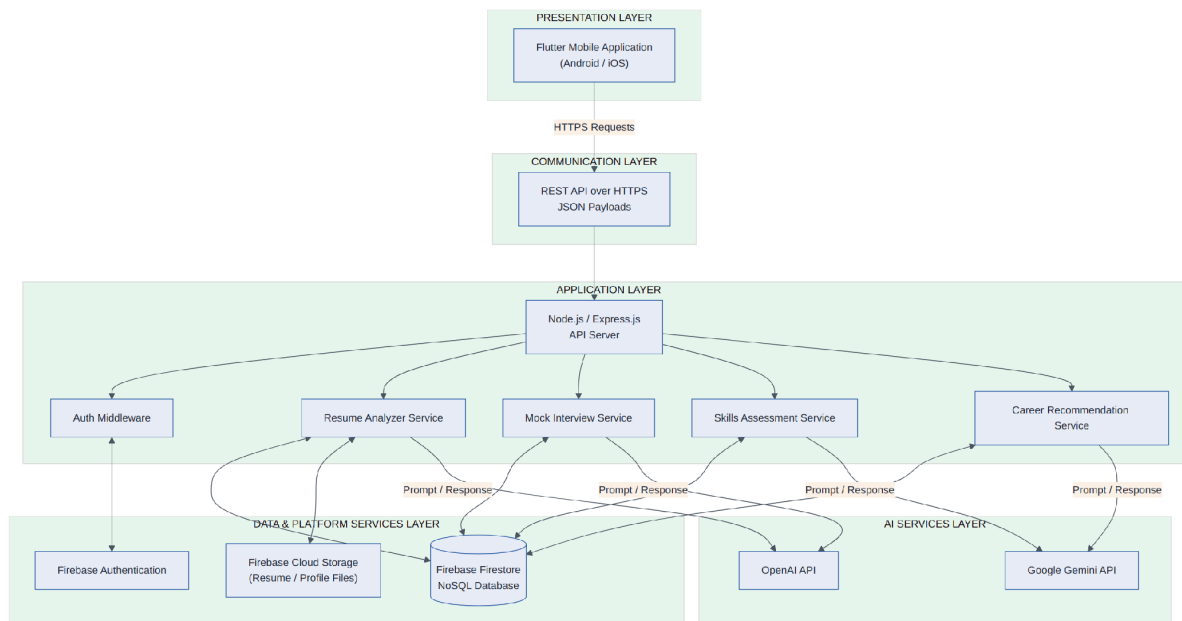


Figure 3.1 — High-Level System Architecture Diagram

#### 3.1 Presentation Layer

The Presentation Layer is a Flutter-based mobile application that runs natively on both Android and iOS. It is responsible for rendering the user interface, capturing user input, managing local application state, and communicating with the backend exclusively through secure HTTPS REST calls. Flutter was selected for its single-codebase cross-platform capability, native-like performance, and rich widget ecosystem, which accelerates UI development while maintaining visual consistency across platforms.

#### 3.2 Application Layer

The Application Layer is implemented using Node.js with the Express.js framework, exposing a RESTful API consumed by the mobile client. This layer houses the core business logic of the system, including authentication middleware, resume-parsing logic, interview-orchestration logic, skills-scoring logic, and the career-recommendation engine. Express.js was chosen for its lightweight, unopinionated structure, its large ecosystem of middleware, and its strong compatibility with the Firebase Admin SDK and third-party AI SDKs.

#### 3.3 Data & Platform Services Layer

Firebase serves as the system's managed backend-as-a-service (BaaS) platform. Firebase Authentication handles secure user identity management (registration, login, token issuance, and verification). Cloud Firestore, a NoSQL document database, stores structured application data such as user profiles, resume metadata, interview sessions, assessment results, and recommendations. Firebase Cloud Storage stores binary assets such as uploaded resume files and profile photos. Using managed Firebase services reduces backend operational overhead and provides built-in scalability, real-time synchronization, and robust security rules.

### 3.4 AI Services Layer

The AI Services Layer integrates with either the OpenAI API or the Google Gemini API to provide the intelligence behind the Resume Analyzer, AI Mock Interview, Skills Assessment feedback, and Career Recommendation modules. The backend acts as a secure intermediary between the mobile client and the AI provider: it constructs structured prompts from user data, sends requests to the AI API, receives generated responses, post-processes them, and returns a clean, structured result to the client. This server-side mediation keeps API keys confidential and allows the system to apply business rules (e.g., scoring normalization) before returning results.

## 4. Application Architecture

Within each of the client and server components, the system follows a layered (n-tier) application architecture that promotes separation of concerns, testability, and maintainability. Figure 4.1 illustrates how each of the four architectural layers is internally structured.

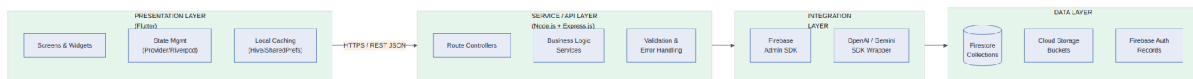


Figure 4.1 — Application (Layered) Architecture Diagram

### 4.1 Flutter Application Structure

| Layer                | Responsibility                                                         | Representative Components                    |
|----------------------|------------------------------------------------------------------------|----------------------------------------------|
| Presentation (UI)    | Renders screens and widgets; captures user gestures and input.         | Screens, Widgets, Themes                     |
| State Management     | Manages application and UI state, coordinates between UI and services. | Provider / Riverpod ViewModels               |
| Service / Repository | Wraps REST API calls and maps JSON responses to Dart model objects.    | ApiService, AuthRepository, ResumeRepository |
| Local Storage        | Caches non-sensitive data for offline viewing and faster reload.       | Hive, SharedPreferences                      |

### 4.2 Backend (Node.js/Express.js) Structure

| Layer                    | Responsibility                                                                          | Representative Components                                       |
|--------------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Route / Controller       | Defines API endpoints and maps HTTP requests to handler functions.                      | authRoutes.js, resumeRoutes.js, interviewRoutes.js              |
| Middleware               | Cross-cutting concerns applied to requests before reaching controllers.                 | authMiddleware.js, validateRequest.js, errorHandler.js          |
| Service / Business Logic | Implements core application logic, independent of HTTP concerns.                        | resumeService.js, interviewService.js, recommendationService.js |
| Integration              | Wraps external SDKs (Firebase Admin, OpenAI/Gemini) behind a stable internal interface. | firebaseAdmin.js, aiClient.js                                   |
| Data Access              | Reads and writes documents to Firestore collections.                                    | userRepository.js, assessmentRepository.js                      |

### 4.3 Design Principles Applied

- Separation of Concerns — UI, business logic, and data access are isolated into distinct layers, easing maintenance and testing.
- Single Responsibility Principle — each service module (e.g., resumeService.js) is responsible for one functional domain.
- Dependency Direction — upper layers depend on lower layers through well-defined interfaces, not the reverse, reducing coupling.
- Statelessness of the API — the Express.js backend does not retain session state between requests; each request is authenticated independently via a Firebase ID token, which supports horizontal scalability.
- Reusability — integration wrappers (e.g., aiClient.js) abstract the specific AI provider so the system can switch between OpenAI and Gemini with minimal code changes.

## 5. Module Design

This section describes the seven core functional modules of the AI-Powered Career Preparation Application. Each module is designed as an independent, cohesive unit with a clearly defined responsibility, a corresponding set of backend service functions, and dedicated Flutter screens. Modules communicate with one another only through the shared Firestore data layer or through the Career Readiness Dashboard, which aggregates their outputs.

### 5.1 User Authentication Module

The User Authentication module manages the secure registration, login, session verification, and logout of student and administrator accounts. It relies on Firebase Authentication for credential management and token issuance, while the Express.js backend verifies Firebase ID tokens on every protected API request through an authentication middleware.

**Key Features**

- Email/password registration and login.
- Google Sign-In as an alternative authentication method.
- Password reset via email verification link.
- Server-side token verification middleware protecting all authenticated routes.
- Automatic session persistence and silent token refresh on the mobile client.

| Attribute       | Details                                                                         |
|-----------------|---------------------------------------------------------------------------------|
| Inputs          | Email address, password, or Google Sign-In OAuth credential.                    |
| Outputs         | Firebase ID token (JWT), authenticated user session, error/validation messages. |
| Technology Used | Firebase Authentication SDK, Firebase Admin SDK, Express.js middleware.         |

**5.2 User Profile Module**

The User Profile module allows students to create and maintain a personal profile containing academic background, career goals, skills, and certifications. This profile data serves as contextual input to the AI-powered modules (Resume Analyzer, Mock Interview, and Career Recommendations), enabling more personalized outputs.

**Key Features**

- Guided profile setup during onboarding.
- Editable profile fields with real-time validation.
- Profile photo upload to Firebase Cloud Storage.
- Skill tags that feed directly into the Skills Assessment and Career Recommendation modules.

| Attribute       | Details                                                                                         |
|-----------------|-------------------------------------------------------------------------------------------------|
| Inputs          | Full name, course/program, year level, career goal, skills list, certifications, profile photo. |
| Outputs         | Persisted profile document in Firestore; profile summary displayed across the app.              |
| Technology Used | Flutter forms, Firebase Cloud Storage, Cloud Firestore.                                         |

**5.3 Resume Analyzer Module**

The Resume Analyzer module allows students to upload a resume file (PDF or DOCX), which the backend parses into plain text and submits to the AI API for evaluation. The AI assesses content quality, structure, keyword relevance, and ATS (Applicant Tracking System) compatibility, returning a numeric score along with specific, actionable improvement suggestions.

**Key Features**

- File upload with format and size validation.
- Server-side text extraction (e.g., pdf-parse, mammoth for DOCX).

- AI-generated scoring across content, structure, and keyword-optimization dimensions.
- Historical view of previous resume versions and score trends.
- Downloadable feedback summary.

| Attribute       | Details                                                                                                     |
|-----------------|-------------------------------------------------------------------------------------------------------------|
| Inputs          | Resume file (PDF/DOCX, max 5 MB), target job role (optional).                                               |
| Outputs         | ATS compatibility score, strengths, weaknesses, and improvement suggestions; stored resume feedback record. |
| Technology Used | Multer (file upload), pdf-parse/mammoth, OpenAI/Gemini API, Cloud Firestore, Cloud Storage.                 |

## 5.4 AI Mock Interview Module

The AI Mock Interview module simulates a realistic job interview by generating role-specific questions, capturing the student's typed or transcribed spoken responses, and providing real-time AI evaluation of each answer. At the conclusion of a session, the module produces a comprehensive performance report covering clarity, confidence indicators, relevance, and overall communication quality.

### Key Features

- Dynamic, context-aware question generation based on job role and prior answers.
- Real-time AI evaluation of answer relevance, clarity, and completeness.
- Adaptive follow-up questioning based on the student's previous response.
- End-of-session performance summary with strengths and improvement areas.
- Session history for tracking improvement over multiple attempts.

| Attribute       | Details                                                                                                          |
|-----------------|------------------------------------------------------------------------------------------------------------------|
| Inputs          | Selected job role/industry, student's answer (text or voice-to-text transcript).                                 |
| Outputs         | Sequential interview questions, per-answer evaluation and feedback, final performance report with overall score. |
| Technology Used | Speech-to-text (device-native or cloud API), OpenAI/Gemini API, Cloud Firestore.                                 |

## 5.5 Skills Assessment Module

The Skills Assessment module evaluates a student's competency across technical domains (e.g., networking, programming, cloud fundamentals) and soft skills (e.g., communication, teamwork, problem-solving) through structured quizzes. Results are scored automatically and, where applicable, enriched with AI-generated explanations for incorrect responses.

### Key Features

- Categorized question bank (technical and soft-skill categories).
- Timed assessment sessions with progress indicators.
- Automatic scoring engine with per-category breakdown.

- AI-generated explanations to reinforce learning after each assessment.
- Administrator interface for managing and updating the question bank.

| Attribute       | Details                                                                                                  |
|-----------------|----------------------------------------------------------------------------------------------------------|
| Inputs          | Selected assessment category, student's answers to multiple-choice/scenario-based items.                 |
| Outputs         | Numeric score per category, percentage breakdown, AI-generated explanations for missed items.            |
| Technology Used | Cloud Firestore (question bank & results), OpenAI/Gemini API (explanations), Express.js scoring service. |

## 5.6 Career Readiness Dashboard Module

The Career Readiness Dashboard aggregates data from the Resume Analyzer, AI Mock Interview, and Skills Assessment modules into a single composite Career Readiness Score. It provides students with a clear, visual summary of their overall preparedness and highlights the specific area (resume, interview, or skills) most in need of improvement.

### Key Features

- Weighted composite scoring algorithm (e.g., Resume 30%, Interview 35%, Skills 35%).
- Visual readiness indicator (progress ring/gauge) and category breakdown cards.
- Trend tracking of readiness score over time.
- Contextual call-to-action directing the student to the module most likely to raise their score.

| Attribute       | Details                                                                                             |
|-----------------|-----------------------------------------------------------------------------------------------------|
| Inputs          | Latest resume score, latest interview score, latest skills assessment scores (read from Firestore). |
| Outputs         | Composite readiness score (0–100), category-level breakdown, personalized "next step" suggestion.   |
| Technology Used | Cloud Firestore aggregation queries, Express.js dashboard service, Flutter charts.                  |

## 5.7 AI Career Recommendations Module

The AI Career Recommendations module analyzes the student's aggregated profile, resume content, and assessment performance to generate a ranked list of suitable job roles, along with a rationale and a set of suggested skills to learn in order to strengthen candidacy for each recommended role.

### Key Features

- AI-generated role matching based on combined profile, resume, and skills data.
- Percentage-based match indicator per recommended role.
- Suggested certifications, courses, or skills to close identified gaps.
- Periodic refresh of recommendations as new resume/assessment data becomes available.

| Attribute       | Details                                                                                                |
|-----------------|--------------------------------------------------------------------------------------------------------|
| Inputs          | User profile data, resume text/keywords, skills assessment results.                                    |
| Outputs         | Ranked list of recommended job roles with match percentage, rationale, and suggested upskilling paths. |
| Technology Used | OpenAI/Gemini API, Cloud Firestore, Express.js recommendation service.                                 |

## 6. Database Design

The system uses Cloud Firestore, a NoSQL document-oriented database, as its primary data store. Although Firestore does not enforce a relational schema, the data model below is presented using an Entity-Relationship (ER) notation to clearly communicate the logical relationships between entities; each entity corresponds to a Firestore collection, and each relationship is implemented through document references (foreign-key-like fields, e.g., uid) rather than SQL joins.

### 6.1 Entity-Relationship Diagram

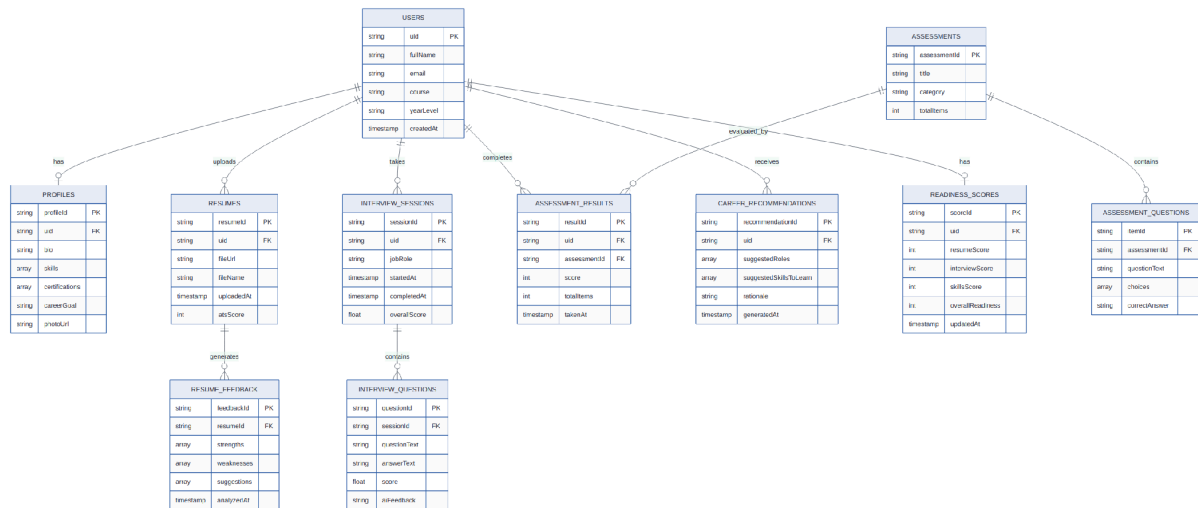


Figure 6.1 — Entity-Relationship Diagram

### 6.2 Entity Descriptions

| Entity   | Description                                                                                                                                         |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| USERS    | Represents a registered account (student or administrator). Serves as the root entity referenced by nearly all other collections via the uid field. |
| PROFILES | Stores extended personal and academic details for a user, including skills, certifications, and career goals; related one-to-one with USERS.        |
| RESUMES  | Stores metadata for each resume file uploaded by a student, including its storage URL and computed ATS score.                                       |

| Entity                 | Description                                                                                                      |
|------------------------|------------------------------------------------------------------------------------------------------------------|
| RESUME_FEEDBACK        | Stores the AI-generated analysis (strengths, weaknesses, suggestions) tied to a specific resume upload.          |
| INTERVIEW_SESSIONS     | Represents one complete AI mock interview attempt, including the target job role and overall score.              |
| INTERVIEW_QUESTIONS    | Stores each individual question, the student's answer, and the corresponding AI evaluation within a session.     |
| ASSESSMENTS            | Represents a skills-assessment definition/category (e.g., "Networking Fundamentals"), managed by administrators. |
| ASSESSMENT_QUESTIONS   | Stores the individual quiz items belonging to a given assessment.                                                |
| ASSESSMENT_RESULTS     | Stores a student's score and performance for a completed assessment attempt.                                     |
| CAREER_RECOMMENDATIONS | Stores the AI-generated list of recommended job roles and suggested skills for a student.                        |
| READINESS_SCORES       | Stores the computed composite Career Readiness Score and its component breakdown for a student.                  |

### 6.3 Database Tables (Firestore Collections)

The subsections below describe the field-level structure of each Firestore collection.

#### 6.3.1 users

| Field     | Type        | Description                                          |
|-----------|-------------|------------------------------------------------------|
| uid       | string (PK) | Unique identifier issued by Firebase Authentication. |
| fullName  | string      | Full name of the user.                               |
| email     | string      | User's email address (unique).                       |
| role      | string      | Account role: "student" or "admin".                  |
| course    | string      | Degree program (e.g., BSIT).                         |
| yearLevel | string      | Academic year level.                                 |
| createdAt | timestamp   | Account creation date/time.                          |

#### 6.3.2 profiles

| Field     | Type          | Description                   |
|-----------|---------------|-------------------------------|
| profileId | string (PK)   | Unique profile document ID.   |
| uid       | string (FK)   | Reference to the owning user. |
| bio       | string        | Short personal summary.       |
| skills    | array<string> | List of self-reported skills. |



| Field          | Type          | Description                            |
|----------------|---------------|----------------------------------------|
| certifications | array<string> | List of certifications/credentials.    |
| careerGoal     | string        | Target job role or industry.           |
| photoUrl       | string        | URL of profile photo in Cloud Storage. |

### 6.3.3 resumes

| Field      | Type        | Description                               |
|------------|-------------|-------------------------------------------|
| resumeld   | string (PK) | Unique resume document ID.                |
| uid        | string (FK) | Reference to the uploading user.          |
| fileUrl    | string      | Cloud Storage URL of the uploaded file.   |
| fileName   | string      | Original file name.                       |
| uploadedAt | timestamp   | Upload date/time.                         |
| atsScore   | number      | Computed ATS compatibility score (0–100). |

### 6.3.4 resume\_feedback

| Field       | Type          | Description                           |
|-------------|---------------|---------------------------------------|
| feedbackId  | string (PK)   | Unique feedback document ID.          |
| resumeld    | string (FK)   | Reference to the associated resume.   |
| strengths   | array<string> | AI-identified resume strengths.       |
| weaknesses  | array<string> | AI-identified resume weaknesses.      |
| suggestions | array<string> | AI-generated improvement suggestions. |
| analyzedAt  | timestamp     | Date/time the analysis was generated. |

### 6.3.5 interview\_sessions

| Field        | Type        | Description                                    |
|--------------|-------------|------------------------------------------------|
| sessionId    | string (PK) | Unique interview session document ID.          |
| uid          | string (FK) | Reference to the interviewing user.            |
| jobRole      | string      | Target job role for the session.               |
| startedAt    | timestamp   | Session start date/time.                       |
| completedAt  | timestamp   | Session completion date/time.                  |
| overallScore | number      | Final AI-computed interview performance score. |

### 6.3.6 interview\_questions (sub-collection of interview\_sessions)

| Field        | Type        | Description                                       |
|--------------|-------------|---------------------------------------------------|
| questionId   | string (PK) | Unique question document ID.                      |
| sessionId    | string (FK) | Reference to the parent interview session.        |
| questionText | string      | AI-generated interview question.                  |
| answerText   | string      | Student's submitted answer.                       |
| score        | number      | AI-evaluated score for this answer.               |
| aiFeedback   | string      | AI-generated qualitative feedback for the answer. |

### 6.3.7 assessments

| Field        | Type        | Description                                   |
|--------------|-------------|-----------------------------------------------|
| assessmentId | string (PK) | Unique assessment document ID.                |
| title        | string      | Assessment title (e.g., "Networking Basics"). |
| category     | string      | Skill category: technical or soft-skill.      |
| totalItems   | number      | Total number of questions in the assessment.  |

### 6.3.8 assessment\_results

| Field        | Type        | Description                             |
|--------------|-------------|-----------------------------------------|
| resultId     | string (PK) | Unique result document ID.              |
| uid          | string (FK) | Reference to the student.               |
| assessmentId | string (FK) | Reference to the taken assessment.      |
| score        | number      | Number of correct answers.              |
| totalItems   | number      | Total items in the assessment.          |
| takenAt      | timestamp   | Date/time the assessment was completed. |

### 6.3.9 career\_recommendations

| Field                  | Type          | Description                                                    |
|------------------------|---------------|----------------------------------------------------------------|
| recommendationId       | string (PK)   | Unique recommendation document ID.                             |
| uid                    | string (FK)   | Reference to the student.                                      |
| suggestedRoles         | array<map>    | List of recommended roles with match percentage and rationale. |
| suggestedSkillsToLearn | array<string> | List of skills/courses recommended to strengthen candidacy.    |
| generatedAt            | timestamp     | Date/time the recommendation was generated.                    |

### 6.3.10 readiness\_scores

| Field            | Type        | Description                                 |
|------------------|-------------|---------------------------------------------|
| scoreId          | string (PK) | Unique readiness score document ID.         |
| uid              | string (FK) | Reference to the student.                   |
| resumeScore      | number      | Latest resume score component.              |
| interviewScore   | number      | Latest interview score component.           |
| skillsScore      | number      | Latest skills assessment score component.   |
| overallReadiness | number      | Weighted composite readiness score (0–100). |
| updatedAt        | timestamp   | Date/time the score was last recalculated.  |

## 7. Process Flow Diagrams

This section presents the process flow models used to design and communicate the system's behavior, from a high-level context view down to detailed sequence-level interactions.

### 7.1 Context Diagram

The Context Diagram (Figure 7.1) presents the AI-Powered Career Preparation Application as a single process and identifies its boundary in relation to external entities: the Student, the System Administrator, the Firebase Platform, and the AI Service Provider (OpenAI/Gemini API).

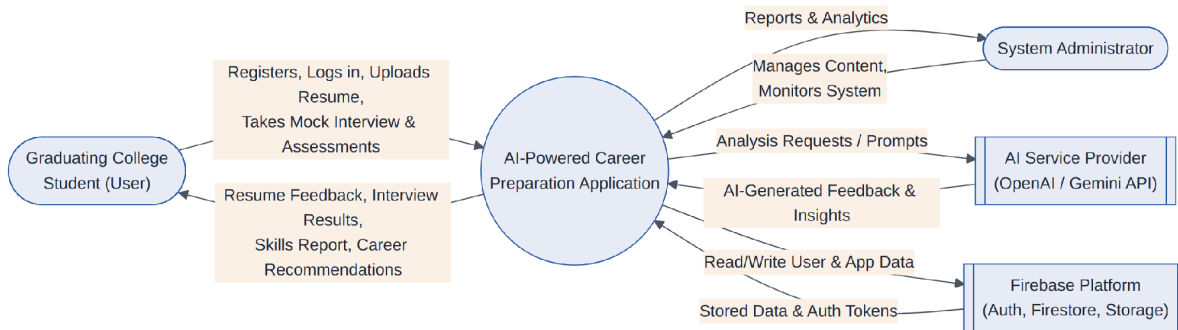


Figure 7.1 — Context Diagram

## 7.2 Data Flow Diagram — Level 0

The Level 0 Data Flow Diagram (Figure 7.2) decomposes the system into its single major process — Career Preparation Management — and shows how data flows between this process, the external entities, and the five primary data stores.

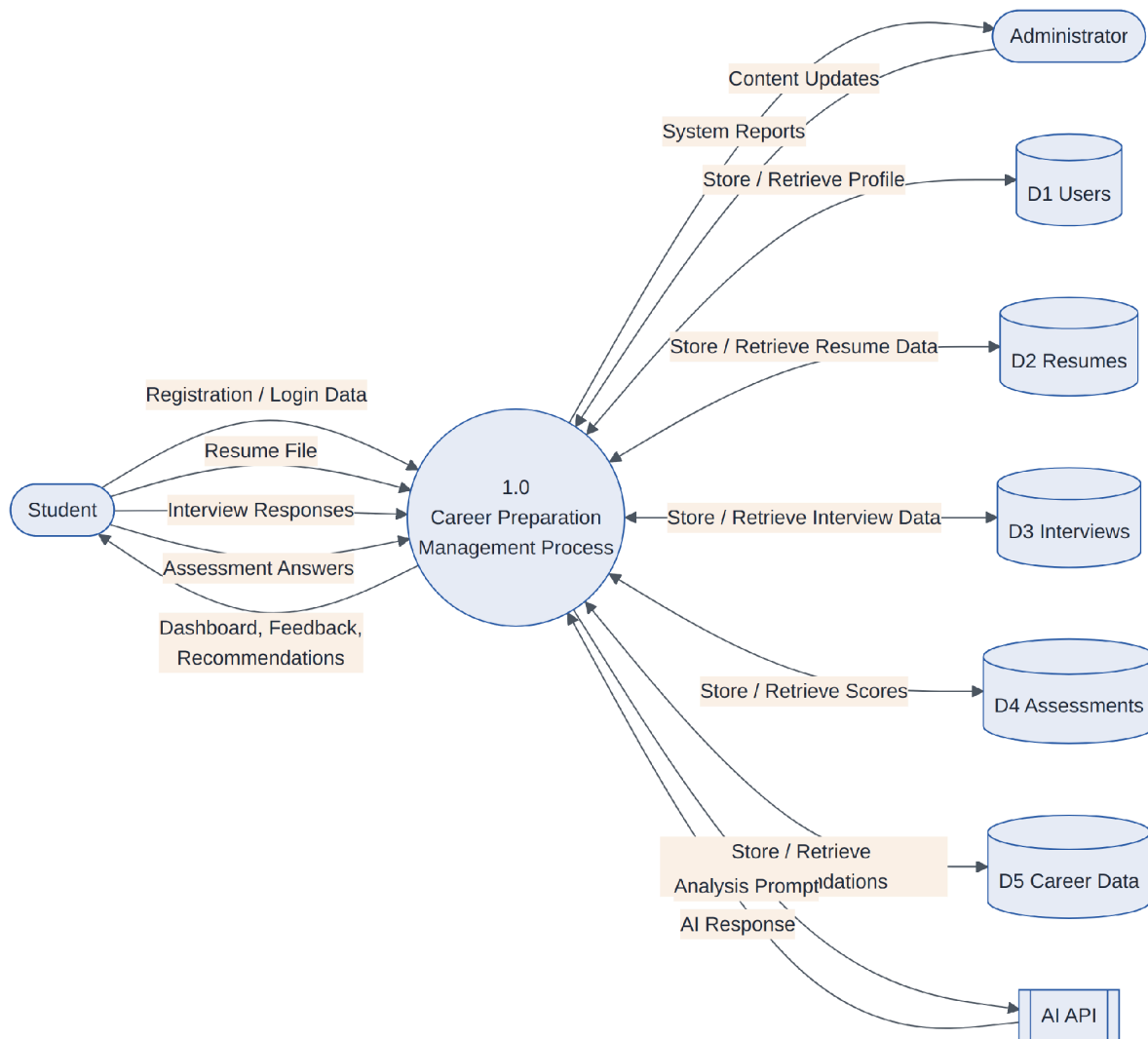


Figure 7.2 — Data Flow Diagram, Level 0

### 7.3 Data Flow Diagram — Level 1

The Level 1 Data Flow Diagram (Figure 7.3) further decomposes the Level 0 process into seven sub-processes corresponding to the system's core modules, illustrating in greater detail how each module interacts with the AI Service and its dedicated data store.

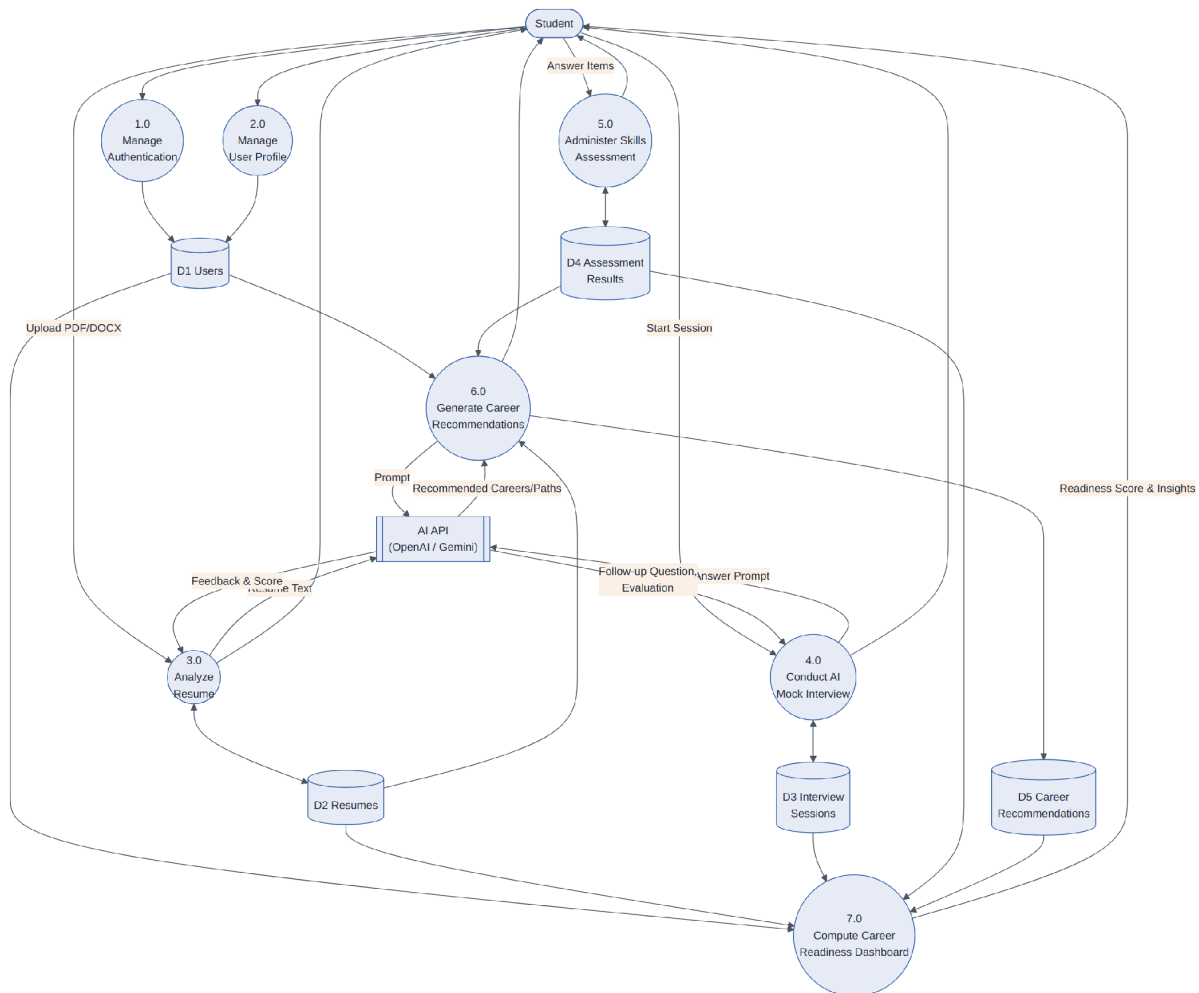


Figure 7.3 — Data Flow Diagram, Level 1

## 7.4 Use Case Diagram

The Use Case Diagram (Figure 7.4) identifies the system's two primary actors — the Student and the Administrator — along with an AI Service actor representing the external AI API, and depicts the principal use cases each actor can perform. The "Generate AI Feedback" use case is modeled as an included behavior shared by the Resume Analyzer, Mock Interview, Skills Assessment, and Career Recommendation use cases, since all four rely on a common AI-invocation flow.

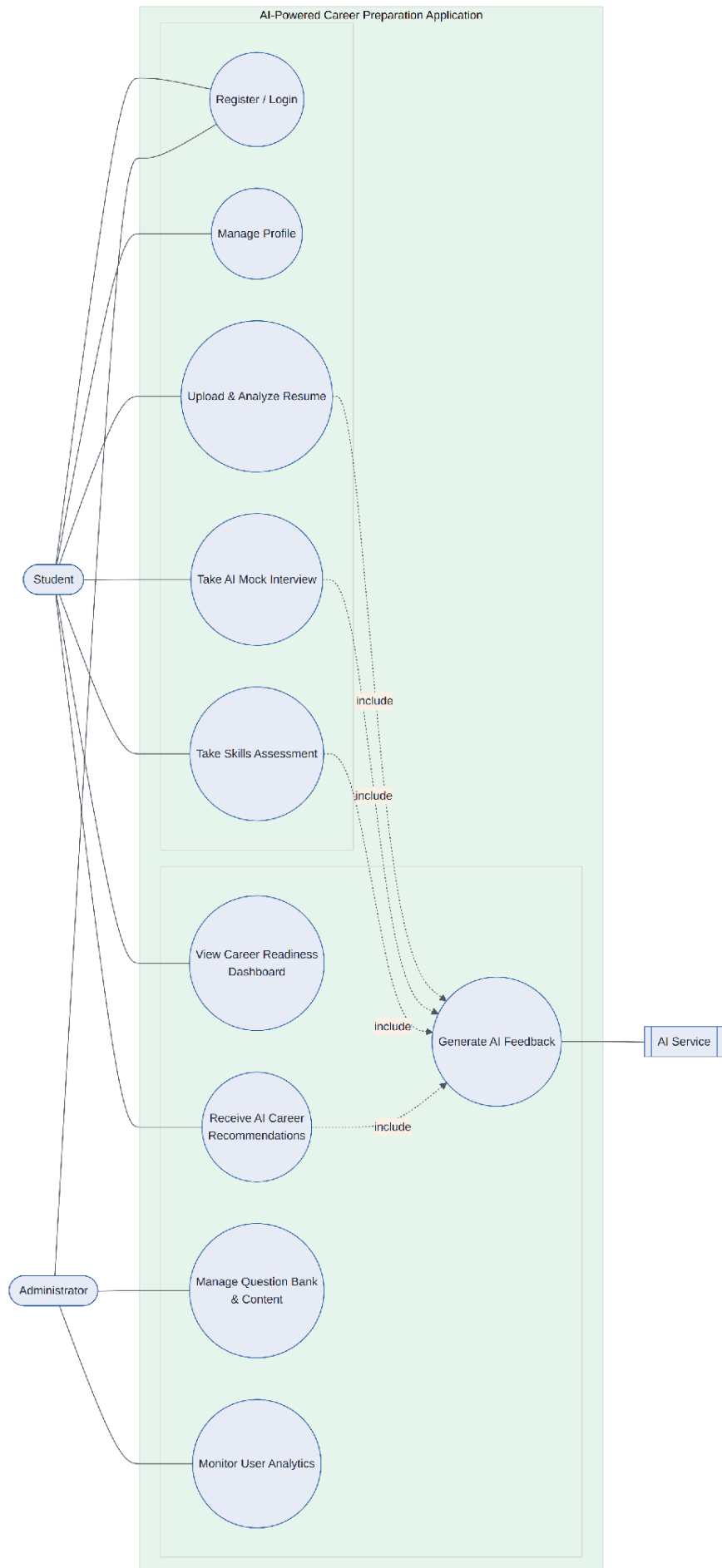


Figure 7.4 — Use Case Diagram

## 7.5 Activity Diagram

The Activity Diagram in Figure 7.5 illustrates the step-by-step workflow of the Resume Analyzer module, from the student logging in and uploading a resume, through server-side validation, AI analysis, and result presentation, including the decision points for invalid file formats and repeat submissions.



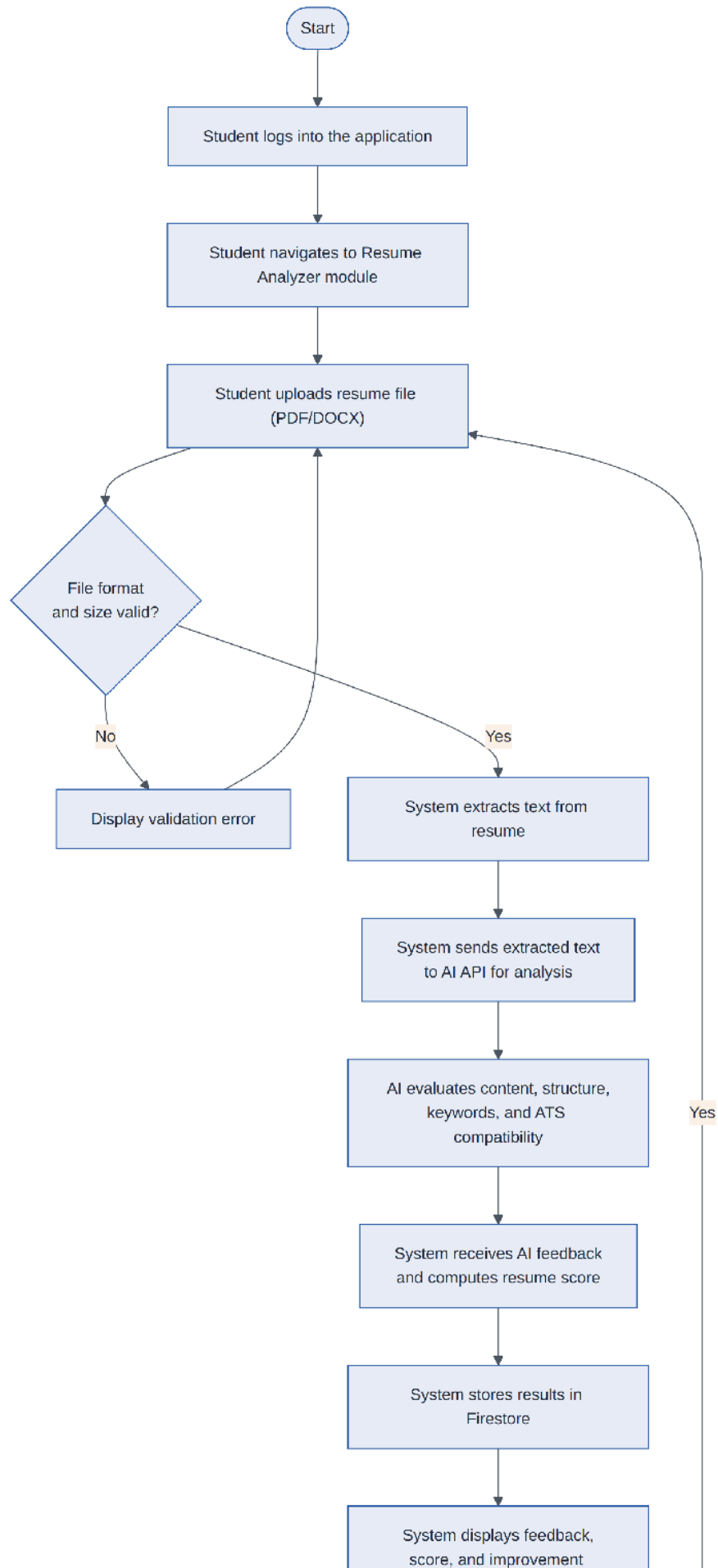


Figure 7.5 — Activity Diagram: Resume Analysis Process

## 7.6 Sequence Diagram

The Sequence Diagram in Figure 7.6 details the interaction sequence among the Student, the Flutter application, the Express.js API, Firebase Authentication, Cloud Firestore, and the AI API during an AI Mock Interview session — from session initiation through iterative question-and-answer exchanges to the generation of the final performance report.

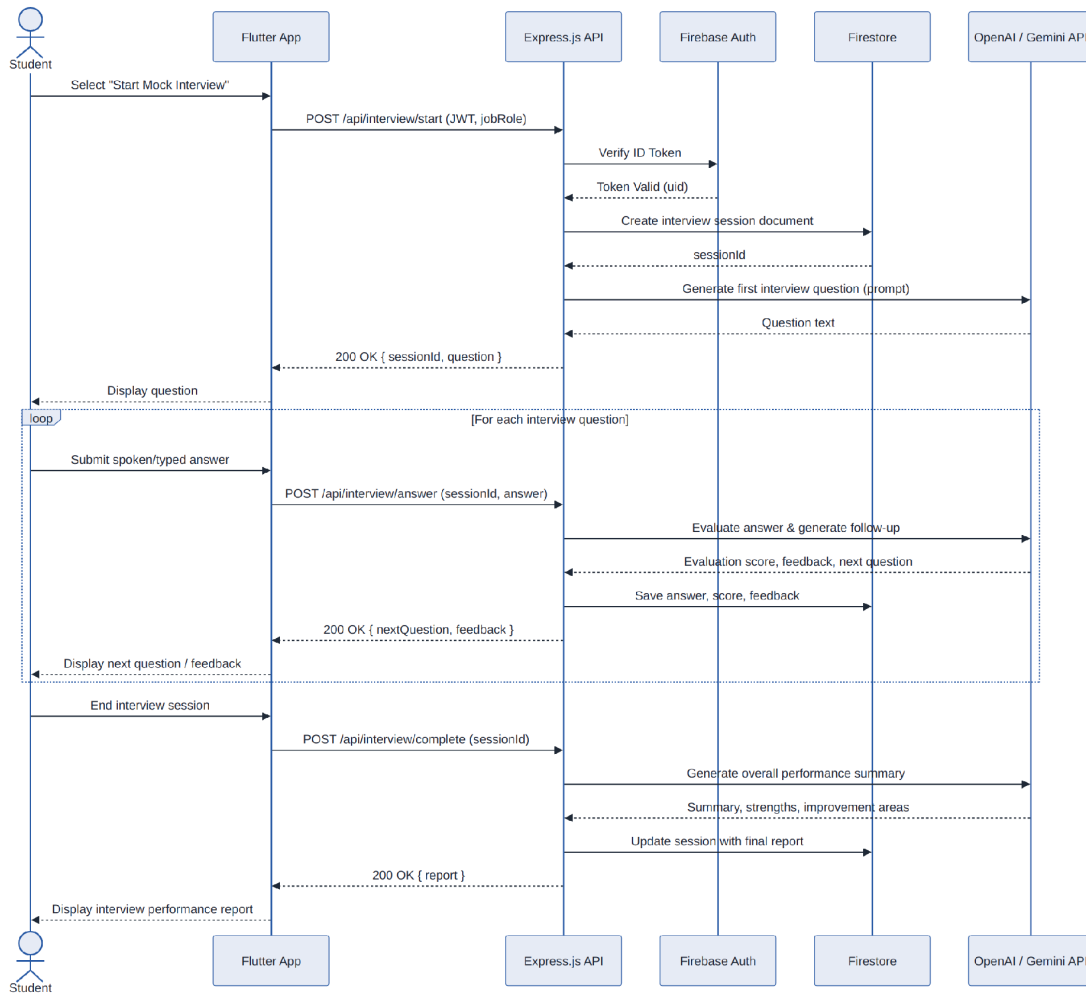


Figure 7.6 — Sequence Diagram: AI Mock Interview Session

## 8. User Interface Design

The user interface of the AI-Powered Career Preparation Application is designed following mobile-first, accessibility-conscious, and minimalist design principles. Screens use a consistent color palette, typography, and component library to reduce cognitive load and reinforce brand identity, while prioritizing legibility and ease of navigation for first-time users.

### 8.1 Screen Descriptions

| Screen                            | Purpose                                                            | Key Elements                                                                             |
|-----------------------------------|--------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Login / Register                  | Authenticate returning users or onboard new users.                 | Email/password fields, Google Sign-In button, forgot-password link, sign-up link.        |
| Career Readiness Dashboard (Home) | Present a consolidated view of the student's overall preparedness. | Readiness score ring, category score cards, recommended next step, bottom navigation.    |
| Resume Analyzer                   | Upload and review AI feedback on a resume.                         | File upload control, ATS score bar, AI feedback list, history of past uploads.           |
| AI Mock Interview                 | Conduct a simulated interview session.                             | Question display, voice/text answer input, live AI evaluation panel, progress indicator. |
| Skills Assessment                 | Administer a categorized skills quiz.                              | Question counter, multiple-choice options, progress bar, next/submit button.             |
| AI Career Recommendations         | Present AI-suggested job roles and upskilling paths.               | Ranked role cards with match percentage, suggested skills panel.                         |
| User Profile                      | View and edit personal, academic, and career information.          | Profile photo, editable fields, skills/certification tags, settings/logout access.       |

### 8.2 Navigation Flow

Figure 8.1 illustrates the overall navigation flow of the application, beginning at the splash screen and branching based on authentication state. Once authenticated, the student lands on the Career Readiness Dashboard, from which all core modules (Resume Analyzer, AI Mock Interview, Skills Assessment, AI Career Recommendations, and Profile) are reachable through a persistent bottom navigation bar, with each module following its own linear sub-flow before returning to the dashboard.

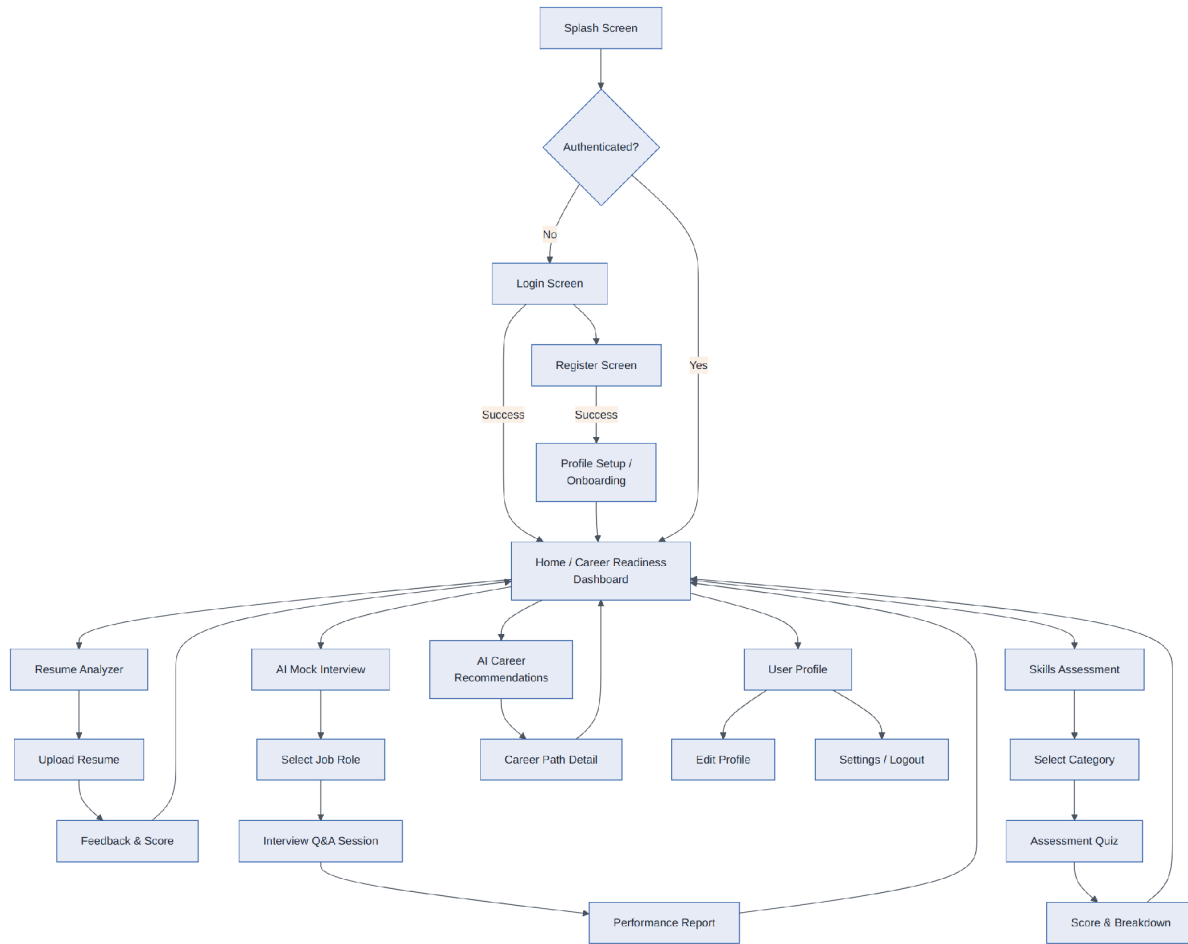


Figure 8.1 — Application Navigation Flow Diagram

### 8.3 Wireframes

Figure 8.2 presents low-fidelity wireframes for six representative screens, illustrating the intended layout, information hierarchy, and primary interactive elements of each screen prior to high-fidelity visual design.

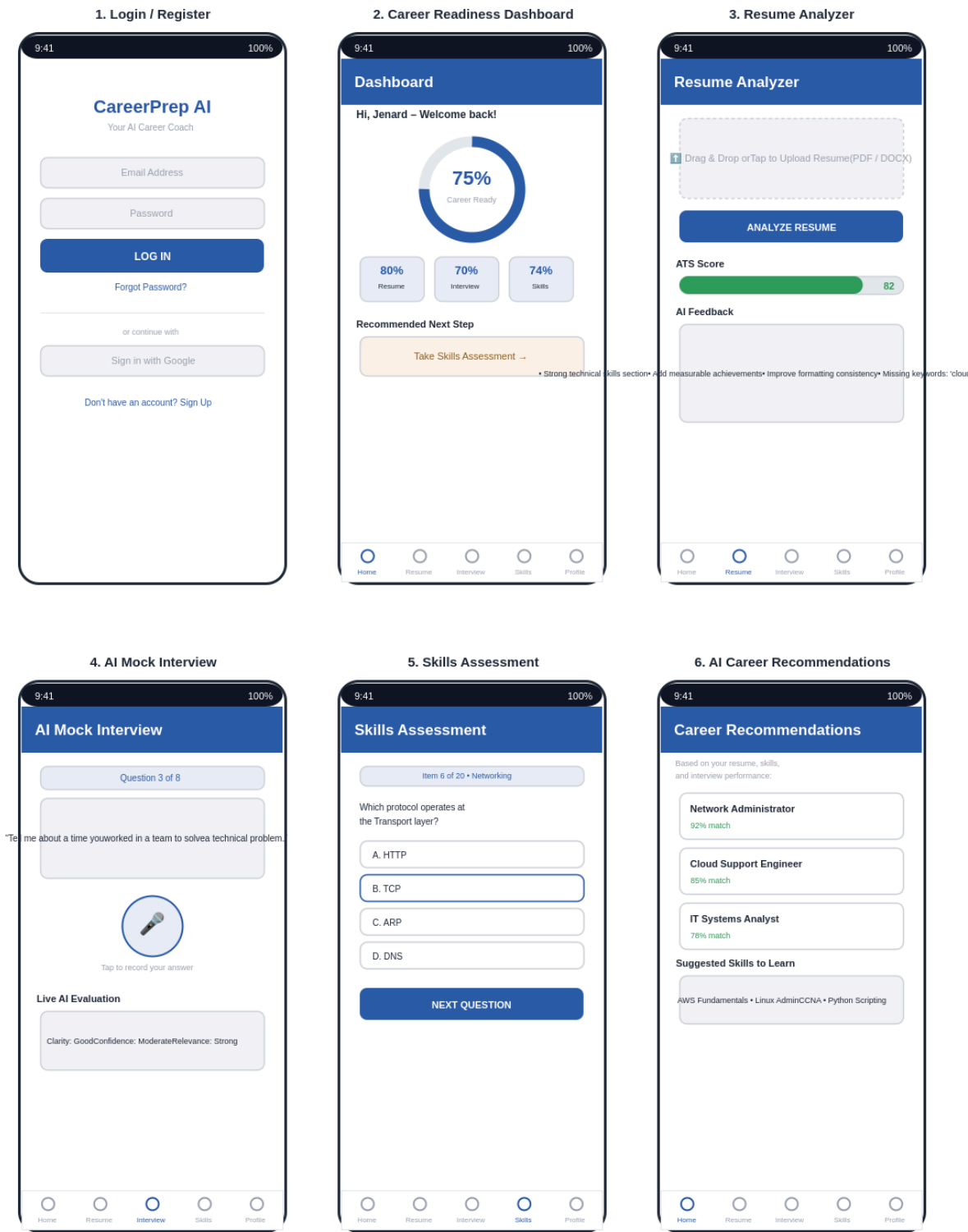


Figure 8.2 — Low-Fidelity Wireframes of Core Application Screens

## 9. API Design

The backend exposes a versioned RESTful API (prefixed with `/api/v1`) that accepts and returns JSON payloads over HTTPS. Every protected endpoint requires a valid Firebase ID token supplied via the `Authorization: Bearer <token>` header, which is verified by the authentication middleware before the request reaches its controller. Standard HTTP status codes (200, 201, 400, 401, 403, 404, 500) are used to communicate outcome, and error responses follow a consistent `{ "error": { "code": ..., "message": ... } }` structure.

### 9.1 Authentication APIs

| Method | Endpoint                                 | Description                                                                            | Auth |
|--------|------------------------------------------|----------------------------------------------------------------------------------------|------|
| POST   | <code>/api/v1/auth/register</code>       | Registers a new student account and creates the corresponding Firestore user document. | No   |
| POST   | <code>/api/v1/auth/login</code>          | Authenticates a user via Firebase and returns a session confirmation payload.          | No   |
| POST   | <code>/api/v1/auth/google</code>         | Authenticates or registers a user via Google Sign-In credential.                       | No   |
| POST   | <code>/api/v1/auth/reset-password</code> | Sends a password-reset email to the specified address.                                 | No   |
| GET    | <code>/api/v1/auth/me</code>             | Retrieves the profile of the currently authenticated user.                             | Yes  |

### 9.2 Resume Analysis API

| Method | Endpoint                                        | Description                                                                 | Auth |
|--------|-------------------------------------------------|-----------------------------------------------------------------------------|------|
| POST   | <code>/api/v1/resumes/upload</code>             | Uploads a resume file (multipart/form-data) and stores it in Cloud Storage. | Yes  |
| POST   | <code>/api/v1/resumes/:resumeld/analyze</code>  | Extracts resume text and submits it to the AI API for scoring and feedback. | Yes  |
| GET    | <code>/api/v1/resumes</code>                    | Retrieves the authenticated user's resume upload history.                   | Yes  |
| GET    | <code>/api/v1/resumes/:resumeld/feedback</code> | Retrieves the AI-generated feedback for a specific resume.                  | Yes  |
| DELETE | <code>/api/v1/resumes/:resumeld</code>          | Deletes a previously uploaded resume and its associated feedback.           | Yes  |

### 9.3 Interview API

| Method | Endpoint                              | Description                                                                   | Auth |
|--------|---------------------------------------|-------------------------------------------------------------------------------|------|
| POST   | /api/v1/interview/start               | Initializes a new AI mock interview session for a selected job role.          | Yes  |
| POST   | /api/v1/interview/:sessionId/answer   | Submits a student's answer and returns AI evaluation plus the next question.  | Yes  |
| POST   | /api/v1/interview/:sessionId/complete | Finalizes the interview session and generates the overall performance report. | Yes  |
| GET    | /api/v1/interview/sessions            | Retrieves the authenticated user's past interview sessions.                   | Yes  |
| GET    | /api/v1/interview/:sessionId          | Retrieves the full detail (questions, answers, scores) of a specific session. | Yes  |

## 9.4 Skills Assessment API

| Method | Endpoint                                    | Description                                                                  | Auth        |
|--------|---------------------------------------------|------------------------------------------------------------------------------|-------------|
| GET    | /api/v1/assessments                         | Lists available assessment categories.                                       | Yes         |
| GET    | /api/v1/assessments/:assessmentId/questions | Retrieves the question set for a selected assessment.                        | Yes         |
| POST   | /api/v1/assessments/:assessmentId/submit    | Submits a student's answers and returns the computed score and breakdown.    | Yes         |
| GET    | /api/v1/assessments/results                 | Retrieves the authenticated user's past assessment results.                  | Yes         |
| POST   | /api/v1/admin/assessments                   | Creates or updates an assessment and its question bank (administrator only). | Yes (Admin) |

## 9.5 Dashboard API

| Method | Endpoint                                  | Description                                                                        | Auth |
|--------|-------------------------------------------|------------------------------------------------------------------------------------|------|
| GET    | /api/v1/dashboard/readiness               | Retrieves the authenticated user's composite Career Readiness Score and breakdown. | Yes  |
| GET    | /api/v1/dashboard/recommendations         | Retrieves the authenticated user's latest AI career recommendations.               | Yes  |
| POST   | /api/v1/dashboard/recommendations/refresh | Triggers regeneration of AI career recommendations using the latest user data.     | Yes  |
| GET    | /api/v1/dashboard/trends                  | Retrieves historical readiness score data for trend visualization.                 | Yes  |

## 9.6 Sample Request/Response — Resume Analysis



The following illustrates a representative request and response for the resume analysis endpoint, showing the expected JSON structure used throughout the API.

| Aspect            | Example                                                                                                                                                                                                |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Request           | POST /api/v1/resumes/r_1029/analyzeAuthorization: Bearer <firebase_id_token>Content-Type: application/jsonBody: { "targetRole": "Network Administrator" }                                              |
| Response (200 OK) | { "atsScore": 82, "strengths": ["Strong technical skills section"], "weaknesses": ["Missing measurable achievements"], "suggestions": ["Add quantifiable results", "Include keywords: cloud, CI/CD"] } |

## 10. Security Design

Because the system handles personally identifiable information (PII) such as student names, academic records, resumes, and career goals, security is treated as a first-class design concern rather than an afterthought. The security design addresses three principal areas: user authentication, data encryption, and secure API communication, supplemented by defense-in-depth practices such as input validation, role-based access control, and Firestore security rules.

### 10.1 User Authentication

All identity management is delegated to Firebase Authentication, a mature, industry-vetted identity platform, rather than implementing custom credential storage. This significantly reduces the system's exposure to common authentication vulnerabilities (e.g., insecure password storage, credential-stuffing susceptibility).

- Passwords are never stored or handled directly by the Express.js backend; Firebase Authentication manages hashing and storage using industry-standard algorithms.
- Every authenticated API request must include a valid Firebase-issued ID token (JWT) in the Authorization header, verified server-side using the Firebase Admin SDK on every request via dedicated middleware.
- ID tokens are short-lived (1 hour by default) and automatically refreshed by the Flutter client using Firebase's refresh-token mechanism, limiting the window of exposure if a token is intercepted.
- Role-Based Access Control (RBAC) distinguishes between "student" and "admin" roles; administrator-only endpoints (e.g., managing the assessment question bank) perform an explicit role check after token verification.
- Optional multi-factor authentication (MFA) via Firebase Authentication is available for administrator accounts to further protect content-management functions.
- Account lockout and rate-limiting policies are applied to login and password-reset endpoints to mitigate brute-force attacks.

### 10.2 Data Encryption

- **Data in Transit** — All communication between the Flutter client, the Express.js API, Firebase services, and the AI API occurs exclusively over HTTPS/TLS 1.2 or higher; plaintext HTTP is disabled at the server and enforced via HSTS headers.
- **Data at Rest** — Cloud Firestore and Cloud Storage automatically encrypt all stored data at rest using Google-managed encryption keys, requiring no additional configuration from the development team.
- **Sensitive Field Handling** — Highly sensitive fields (e.g., API keys, service account credentials) are never stored in the database or client code; they are managed exclusively through environment variables and a secrets manager on the server.
- **File Upload Integrity** — Uploaded resume files are scanned for file-type conformity (magic-byte validation, not just file extension) before processing, and are stored with restricted access permissions in Cloud Storage.

### 10.3 Secure API Communication

- All REST endpoints are served over HTTPS; the Express.js server redirects any HTTP request to HTTPS.
- Cross-Origin Resource Sharing (CORS) is explicitly configured to accept requests only from the application's known client origins.
- Request payloads are validated and sanitized (e.g., using a schema-validation library such as Joi or express-validator) before being processed, mitigating injection and malformed-payload attacks.
- Firestore Security Rules are configured to deny direct client-side reads/writes to sensitive collections, ensuring that all data mutations pass through the validated, authenticated Express.js API rather than being written directly by the mobile client.
- API keys for the OpenAI/Gemini API are stored server-side only and are never exposed to or bundled within the Flutter client, preventing key extraction from the compiled mobile application.
- Standardized error handling avoids leaking internal stack traces or implementation details to the client; detailed errors are logged server-side only.
- Rate limiting (e.g., via express-rate-limit) is applied per-user and per-IP on AI-invoking endpoints to prevent abuse and to control third-party AI API costs.
- Dependency and vulnerability scanning (e.g., npm audit, Dependabot) is performed regularly to identify and patch known vulnerabilities in third-party packages.

### 10.4 Privacy and Data Governance Considerations

In alignment with the Philippine Data Privacy Act of 2012 (Republic Act No. 10173), the system incorporates the following data governance practices:

- Explicit user consent is obtained during registration for the collection and AI-based processing of resume and profile data.
- Students may request the deletion of their account and associated data, which triggers a cascading removal of related Firestore documents and Cloud Storage files.
- Data collected is limited to what is necessary for the system's stated career-preparation purpose (data minimization principle).

- Access to student data by administrators is limited to aggregate, anonymized analytics wherever individual-level detail is not required.

## 11. Deployment Architecture

The system is designed for cloud-based deployment to maximize availability, scalability, and ease of maintenance, while minimizing infrastructure management overhead for the capstone team. Figure 11.1 illustrates the target deployment topology.

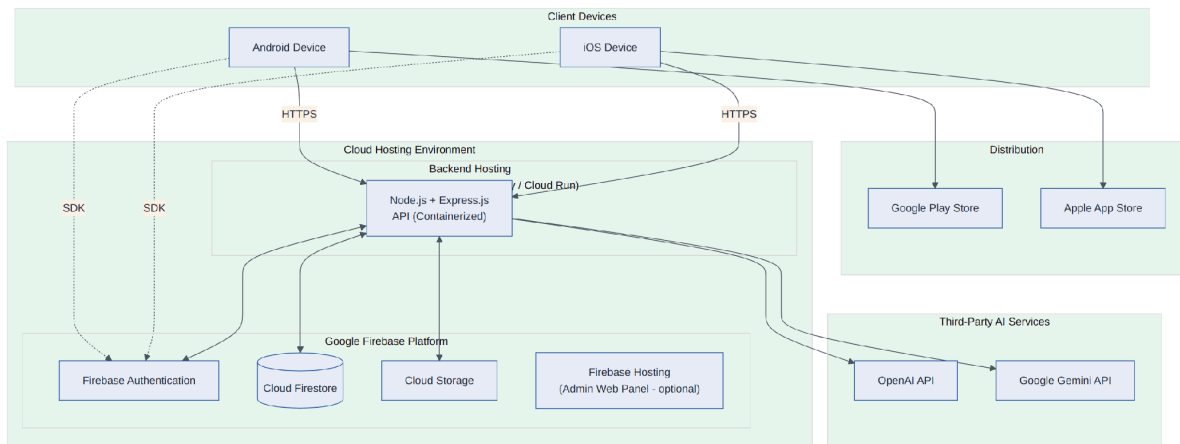


Figure 11.1 — Deployment Architecture Diagram

### 11.1 Client Distribution

The Flutter application is compiled into platform-specific release binaries — an Android App Bundle (.aab) for distribution via the Google Play Store, and an .ipa build for distribution via the Apple App Store (or, for capstone demonstration purposes, via ad-hoc/TestFlight distribution). End users install the application directly from their respective device app stores.

### 11.2 Backend Hosting

The Node.js/Express.js API is containerized using Docker and deployed to a managed cloud platform such as Render, Railway, or Google Cloud Run. Containerization ensures a consistent runtime environment across development, staging, and production, and allows the backend to scale horizontally (additional container instances) in response to increased load, such as during peak usage periods near graduation season.

### 11.3 Firebase Platform Services

Firebase Authentication, Cloud Firestore, and Cloud Storage are fully managed Google Cloud services requiring no server provisioning. These services scale automatically and provide built-in high availability across Google's global infrastructure. An optional Firebase Hosting instance may be used to serve a lightweight web-based administrative panel.

## 11.4 AI Service Integration

The backend communicates with the OpenAI API or Google Gemini API over outbound HTTPS calls. No AI model infrastructure is self-hosted; the system instead consumes these providers' managed inference endpoints, which reduces operational complexity while introducing a dependency on third-party service availability — a risk mitigated through request timeouts, retry logic, and graceful degradation (e.g., cached/previous results) in the mobile client.

## 11.5 Environments

| Environment | Purpose                                                               | Notes                                                               |
|-------------|-----------------------------------------------------------------------|---------------------------------------------------------------------|
| Development | Local development and unit testing.                                   | Firebase Emulator Suite used to simulate Auth/Firestore locally.    |
| Staging     | Integration testing and sprint review demonstrations.                 | Mirrors production configuration with a separate Firebase project.  |
| Production  | Live environment for end users following capstone deployment/defense. | Isolated Firebase project, production API keys, monitoring enabled. |

# 12. Hardware and Software Requirements

## 12.1 Client-Side (Mobile Device) Requirements

| Requirement                | Minimum Specification                                                                   |
|----------------------------|-----------------------------------------------------------------------------------------|
| Operating System (Android) | Android 8.0 (Oreo) or later                                                             |
| Operating System (iOS)     | iOS 13 or later                                                                         |
| RAM                        | 2 GB minimum (3 GB or higher recommended)                                               |
| Storage                    | At least 150 MB free space for application installation                                 |
| Network                    | Active internet connection (Wi-Fi or mobile data) required for AI-powered features      |
| Microphone                 | Required for voice-based mock interview responses (optional; text input also supported) |

## 12.2 Development Environment Requirements

| Component             | Specification / Tool                          |
|-----------------------|-----------------------------------------------|
| Frontend Framework    | Flutter SDK (stable channel, latest version)  |
| Programming Languages | Dart (frontend), JavaScript/Node.js (backend) |
| Backend Framework     | Node.js (LTS version) with Express.js         |

| Component        | Specification / Tool                      |
|------------------|-------------------------------------------|
| Database         | Firebase Cloud Firestore                  |
| Authentication   | Firebase Authentication                   |
| File Storage     | Firebase Cloud Storage                    |
| AI Integration   | OpenAI API or Google Gemini API           |
| IDE / Editors    | Visual Studio Code, Android Studio        |
| Version Control  | Git and GitHub                            |
| API Testing      | Postman or Insomnia                       |
| Design Tools     | Figma (UI/UX wireframing and prototyping) |
| Containerization | Docker (for backend deployment)           |

### 12.3 Server / Hosting Requirements

| Component                       | Minimum Specification                                                         |
|---------------------------------|-------------------------------------------------------------------------------|
| Compute (Backend API Container) | 1 vCPU, 512 MB–1 GB RAM (auto-scaling enabled)                                |
| Database                        | Cloud Firestore (serverless, auto-scaling; no fixed provisioning required)    |
| Storage                         | Cloud Storage bucket with sufficient capacity for resume/profile file uploads |
| Bandwidth                       | Sufficient outbound bandwidth to support concurrent AI API requests           |
| SSL/TLS Certificate             | Required for all HTTPS endpoints (typically provided by the hosting platform) |

## 13. Testing Plan

Testing is conducted continuously throughout each Agile sprint rather than as a single terminal phase, in order to detect defects early and validate that each increment meets its acceptance criteria. The testing strategy combines multiple levels of testing, described below, culminating in a full user-acceptance testing (UAT) cycle prior to final deployment.

### 13.1 Levels of Testing

| Test Level   | Objective                                                                                                               | Responsible Party | Tools                                           |
|--------------|-------------------------------------------------------------------------------------------------------------------------|-------------------|-------------------------------------------------|
| Unit Testing | Verify that individual functions/components (e.g., scoring algorithms, validation logic) behave correctly in isolation. | Developers        | Jest / Mocha (backend), Flutter test (frontend) |

| Test Level                    | Objective                                                                                                    | Responsible Party                      | Tools                                    |
|-------------------------------|--------------------------------------------------------------------------------------------------------------|----------------------------------------|------------------------------------------|
| Integration Testing           | Verify correct interaction between modules, e.g., Resume Analyzer service ↔ AI API ↔ Firestore.              | Developers / QA                        | Postman, Supertest                       |
| System Testing                | Validate end-to-end functionality of the complete application against functional requirements.               | QA Team                                | Manual test scripts, Postman collections |
| User Acceptance Testing (UAT) | Confirm the system meets real user needs and expectations.                                                   | Graduating student volunteers, adviser | Structured UAT scripts, feedback forms   |
| Performance Testing           | Assess API response times and system behavior under concurrent load, particularly for AI-invoking endpoints. | Developers                             | Apache JMeter / k6                       |
| Security Testing              | Verify authentication enforcement, input validation, and data-access restrictions.                           | Developers / QA                        | OWASP ZAP, manual penetration checks     |
| Usability Testing             | Evaluate ease of navigation, clarity of AI feedback presentation, and overall user experience.               | Sample end users                       | Moderated usability sessions, surveys    |

## 13.2 Sample Test Cases

| Test Case ID | Module              | Test Scenario                                              | Expected Result                                                           |
|--------------|---------------------|------------------------------------------------------------|---------------------------------------------------------------------------|
| TC-01        | User Authentication | Register with a valid, unused email and a strong password. | Account is created; user is redirected to profile onboarding.             |
| TC-02        | User Authentication | Attempt login with an incorrect password.                  | System displays an appropriate error message; access is denied.           |
| TC-03        | Resume Analyzer     | Upload a resume file exceeding the 5 MB size limit.        | System rejects the upload and displays a file-size validation error.      |
| TC-04        | Resume Analyzer     | Upload a valid PDF resume for a Networking-track student.  | System returns an ATS score and at least one strength and one suggestion. |
| TC-05        | AI Mock Interview   | Submit an empty answer to an interview question.           | System prompts the user to provide an answer before proceeding.           |
| TC-06        | AI Mock Interview   | Complete a full interview session (8 questions).           | System generates a final performance report with an overall score.        |
| TC-07        | Skills Assessment   | Submit answers for a 20-item assessment.                   | System computes and displays an accurate score and percentage breakdown.  |

| Test Case ID | Module                     | Test Scenario                                                          | Expected Result                                                                  |
|--------------|----------------------------|------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| TC-08        | Career Readiness Dashboard | View dashboard after completing resume, interview, and skills modules. | Composite readiness score reflects the weighted average of all three components. |
| TC-09        | Security                   | Call a protected API endpoint without an Authorization header.         | API responds with HTTP 401 Unauthorized.                                         |
| TC-10        | Security                   | Student account attempts to access an admin-only endpoint.             | API responds with HTTP 403 Forbidden.                                            |

### 13.3 Acceptance Criteria

A sprint deliverable or module is considered accepted when it satisfies all of the following conditions:

- All associated unit and integration test cases pass with no critical or high-severity defects outstanding.
- The feature has been demonstrated in a sprint review and validated against its original user story and acceptance criteria.
- The feature meets baseline non-functional requirements — e.g., API responses returned within an acceptable time threshold (typically under 3 seconds for non-AI calls, and under 10 seconds for AI-generated responses).
- No unresolved security vulnerabilities of medium severity or higher are present in the affected module.
- The corresponding UI has been reviewed for consistency with the approved wireframes and design guidelines.

### 13.4 Defect Severity Classification

| Severity | Description                                                        | Example                                                      |
|----------|--------------------------------------------------------------------|--------------------------------------------------------------|
| Critical | System crash, data loss, or complete failure of a core feature.    | Application crashes when submitting a mock interview answer. |
| High     | Major feature does not work as intended but system remains usable. | Resume score is not saved after analysis completes.          |
| Medium   | Minor functional issue with an available workaround.               | Skills assessment timer does not reset between attempts.     |
| Low      | Cosmetic or usability issue with minimal functional impact.        | Misaligned button on the profile screen.                     |

## 14. Future Enhancements

---

While the current scope of the system addresses the core needs of graduating students preparing for employment, several enhancements have been identified as valuable directions for future development beyond the capstone timeline:

- **Video-Based Mock Interviews** — Incorporating camera-based analysis (e.g., eye contact, posture, and facial expression cues) to provide more comprehensive non-verbal communication feedback.
- **Employer/Job-Board Integration** — Connecting the Career Recommendation module with real job postings from partner companies or public job boards (e.g., LinkedIn, JobStreet) to bridge preparation with actual application opportunities.
- **Offline AI Feedback Caching** — Allowing previously generated AI feedback to be fully accessible offline, with queued analysis requests processed once connectivity is restored.
- **Peer Benchmarking** — Providing anonymized, aggregate comparisons of a student's readiness score against peers within the same program or institution.
- **Gamification** — Introducing achievement badges, streaks, and milestones to increase student engagement with the resume, interview, and skills-assessment modules.
- **Multi-language Support** — Extending the AI Mock Interview and Resume Analyzer modules to support Filipino and other regional languages.
- **Institutional Analytics Portal** — Building a web-based dashboard for career-services offices to view aggregate readiness trends across a graduating cohort, supporting data-driven curriculum and career-counseling decisions.
- **Adaptive Learning Recommendations** — Integrating with online learning platforms (e.g., Coursera, Udemy) to recommend and track specific courses that address a student's identified skill gaps.
- **Alumni Success Tracking** — Following up with graduates post-employment to correlate readiness scores with actual employment outcomes, enabling continuous refinement of the AI scoring models.